

Planificación de Implementación vs. Desarrollo: Un Modelo Mental al Que Siempre Vuelvo

Una reflexión práctica sobre cómo la planificación de implementación puede reducir el esfuerzo de desarrollo, sin pretender que el gráfico sea un modelo científico.

Publicado: 9 de junio de 2026 · Actualizado: 5 de septiembre de 2026

Artículo original: <https://www.cleisson.com/es-ES/blog/implementation-planning-vs-developing>

En el proyecto en el que trabajo, empezamos a crear tareas separadas para planes de implementación. En algunos casos, la tarea de desarrollo solo se creaba, refinaba o dividía después de terminar la tarea de planificación.

Al principio, quería ir directamente al código. Ahí es donde las cosas se sienten concretas. Pero, después de ver este proceso unas cuantas veces, empecé a valorar el tiempo para entender qué afectaría un cambio antes de elegir un enfoque. En funcionalidades, refactorizaciones, integraciones y migraciones, llegar a entenderlo puede dar más trabajo del que sugiere el ticket.

Mi regla es planificar mientras sea la forma menos costosa de responder a una pregunta que pueda cambiar la implementación. Cuando un experimento pequeño y reversible pueda enseñarnos más, es hora de construir. Evalúo el plan por el esfuerzo y el riesgo que ahorra a lo largo de todo el cambio, incluido el tiempo dedicado a planificar.

Por planificación de implementación, entiendo leer el código existente, comprender los límites del sistema, comprobar supuestos y definir la estructura, las pruebas y los pasos manuales que necesita el cambio. El desarrollo incluye escribir código, ajustar pruebas, revisar, integrar, depurar, desplegar y resolver lo que descubrimos por el camino.

Estas actividades se solapan. Un pequeño spike de implementación puede formar parte de la planificación, y el código funcionando puede demostrar que un plan estaba equivocado. Las tareas separadas ayudan a organizar el trabajo; no hacen que el aprendizaje avance en línea recta.

Por qué una tarea separada de planificación puede ayudar

Sin una etapa explícita de planificación, la investigación suele quedar repartida entre comentarios en tickets, hilos de Slack y descubrimientos durante el desarrollo. Para un cambio pequeño, eso puede bastar. Para uno mayor, quiero que esos hallazgos estén en un lugar donde el equipo pueda usarlos antes de que el trabajo dependa de un supuesto sin comprobar.

Una tarea dedicada hace visible esa investigación. Da al ingeniero tiempo para leer el código, preguntar quién es responsable de un flujo y comprobar si un patrón existente resuelve parte del problema. El resultado puede ser una nota breve con las áreas afectadas, una secuencia de implementación y los riesgos que el equipo necesita discutir.

La tarea de desarrollo que viene después puede tener límites y expectativas de pruebas más claros. También puede incluir esos pasos manuales aburridos que es fácil olvidar hasta el día del release.

Yo ajustaría el plan a las consecuencias de equivocarnos con el cambio. Una modificación local y fácil de deshacer quizá solo necesite una nota en el ticket. Una migración con dependencias y comportamientos heredados merece una investigación más cuidadosa. Una tarea separada de planificación es útil cuando da espacio a ese trabajo; no la haría obligatoria para cada cambio de código.

Qué quiero que un plan descubra

Un ticket rara vez contiene toda la historia de un sistema. Las reglas de negocio, las concesiones antiguas y las áreas marcadas como *"por favor no toques esto a menos que sepas por qué"* pueden cambiar el enfoque.

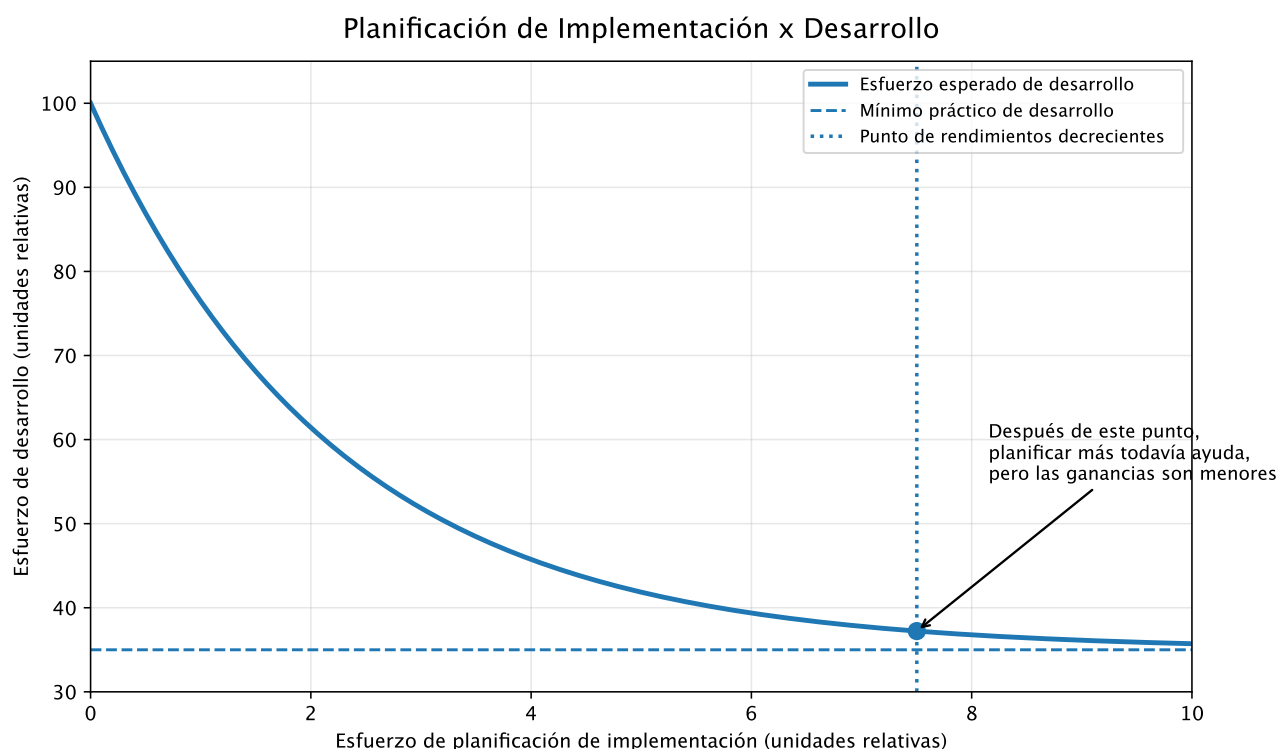
Quiero que el plan responda a unas cuantas preguntas prácticas:

- ¿Qué va a cambiar, qué queda fuera del alcance y qué sistemas, módulos, jobs, eventos o APIs se verán afectados? ¿Quién es responsable de los flujos implicados?
- ¿Qué reglas de negocio y comportamientos heredados deben mantenerse? ¿Qué patrones de código existentes debemos seguir o evitar?
- ¿Qué estructura y secuencia de implementación tienen sentido, y qué supuestos podrían obligarnos a cambiarlas?
- ¿Qué pruebas y casos límite nos dirán si el cambio funciona? ¿Qué podemos automatizar y qué sigue necesitando verificación humana?
- ¿Qué debe ocurrir alrededor del release, incluidos scripts, migraciones, backfills, feature flags, configuración y orden de despliegue? ¿Quién se encarga de los pasos manuales, qué comprobamos después y cómo nos recuperamos si algo sale mal?

Las respuestas pueden caber en un ticket. Para un cambio complicado, algunas quizá necesiten su propia investigación. Me importa más que ayuden al siguiente ingeniero a tomar una decisión que la cantidad de documentación que produzcan.

Qué explica bien la curva

Así imagino la relación entre una planificación útil y el esfuerzo de desarrollo que viene después:



Modelo ilustrativo — basado en observación práctica, no en un análisis formal de datos.

Las unidades son ilustrativas. Este es un modelo mental basado en la experiencia, no un estudio ni un benchmark.

La caída pronunciada del principio representa el trabajo evitable que puede revelar una investigación breve. Podemos descubrir que un flujo ya existe, que un campo tiene un significado de negocio oculto o que una dependencia se comporta de forma diferente en producción. Entender por qué un cambio parecido causó problemas puede alterar el enfoque antes de que tengamos mucho código que deshacer. Una hora buscando una solución existente puede ahorrar horas implementando una sustituta.

Ese es el tipo de ahorro que tengo en mente: menos descubrimientos tardíos, menos idas y vueltas sobre quién es responsable de un flujo y menos *"ah, esto también afecta a aquel otro servicio"*.

Pero parte del trabajo solo se adelanta. Leer un módulo durante la planificación, en vez de durante el desarrollo, sigue llevando tiempo. Una tarea de desarrollo más corta, por sí sola, no demuestra que el equipo haya ahorrado esfuerzo. El ahorro aparece cuando esa comprensión anticipada evita trabajo que tendríamos que rehacer o nos ayuda a elegir una implementación más sencilla.

En algún momento, la curva se aplana. Alguien todavía tiene que escribir, probar, revisar, integrar y desplegar el cambio. Algunas preguntas también necesitan código funcionando para poder responderlas. Un documento más largo puede hacernos sentir más seguros sin resolver esas preguntas.

El coste que falta en el gráfico

Una forma de esbozar la curva es la ecuación a la que siempre vuelvo:

$$\text{Desarrollo}(P) = D_{\min} + (D_0 - D_{\min}) * e^{(-kP)}$$

Aquí, P es el esfuerzo de planificación y $\text{Desarrollo}(P)$ es el esfuerzo esperado de desarrollo posterior. D_0 es el esfuerzo de desarrollo sin planificación, mientras que $D_{\min}$ es el mínimo práctico que queda incluso después de una planificación útil. El parámetro k controla la rapidez con la que la curva se acerca a ese mínimo; representa cuánto reduce la planificación la incertidumbre y el trabajo que habría que rehacer.

Con $D_0 > D_{\min}$ y $k > 0$, la curva siempre baja. Mirarla de forma aislada hace parecer que más planificación siempre es mejor. Pero planificar también tiene un coste.

Para un alcance fijo, con planificación y desarrollo medidos en las mismas unidades de esfuerzo, la cuenta más completa es $\text{Total}(P) = P + \text{Desarrollo}(P)$. Cada actividad entra en un solo lado de esa suma: un spike exploratorio contabilizado como planificación no debe contarse también como desarrollo.

En este modelo simplificado, una hora más de planificación solo reduce el esfuerzo total si ahorra más de una hora después. Cuando el ahorro cae por debajo de eso, la planificación adicional aumenta el total, aunque la estimación de desarrollo siga bajando. En un cambio que ya entendemos bien, la planificación adicional puede no compensar el esfuerzo dedicado. Esto da una consecuencia práctica al aplanamiento de la curva: necesitamos un motivo para seguir planificando.

No usaría esta ecuación para estimar un sprint ni calcular un porcentaje de planificación. No tengo valores medidos para estos parámetros. Presupone una planificación útil y un alcance fijo, mientras que la planificación real puede revelar trabajo que faltaba y aumentar una estimación. Ese descubrimiento puede evitar un release incompleto; la estimación menor habría sido engañosa.

El esfuerzo también es solo una parte de la decisión. En un cambio con consecuencias graves en caso de fallo, dedicaría más tiempo a validar la recuperación, aunque no acertara la implementación. Un plan sigue teniendo que cumplir las necesidades de seguridad y corrección del cambio.

Elige el siguiente paso por lo que puede enseñarte

Pensemos en una migración hipotética de un campo. Antes de cambiar el schema, querría saber qué jobs y servicios dependen de ese campo, qué comportamientos heredados deben mantenerse y qué orden de despliegue permitiría que esos consumidores siguieran funcionando. Las respuestas podrían cambiar la secuencia de implementación.

Ahora supongamos que la duda pendiente es cómo se comportará un backfill con datos representativos. Una prueba acotada en un entorno seguro puede enseñarnos más que otra discusión de diseño. El plan puede definir qué probar y qué resultado nos haría reconsiderar el enfoque. Después podemos ejecutar el experimento y revisar el plan.

El mismo cambio puede necesitar más investigación en un área y código funcionando en otra. Dedicaría más esfuerzo de planificación a una decisión difícil de revertir y usaría pequeños experimentos donde podamos aprender sin comprometer el resto del sistema.

Por eso también revisaría el plan durante el desarrollo. Cuando la implementación revela un supuesto equivocado, actualizar el plan ayuda a que la siguiente tarea aproveche lo aprendido. Seguir la secuencia original solo porque la tarea de planificación está cerrada perdería el sentido.

Cómo decido que hemos planificado lo suficiente

He planificado tanto de menos como de más. La presión por avanzar rápido puede dejar preguntas básicas sobre impacto, pruebas o rollout que acaben interrumpiendo el desarrollo. El deseo de evitar errores puede mantenernos buscando en un documento respuestas que necesitan un experimento.

Para mí, el punto ideal sigue estando en algún lugar entre caos y teatro. Quiero que el equipo pueda decir cuál es el siguiente cambio pequeño y acotado, explicar por qué tiene sentido empezar por él y describir cómo vamos a comprobarlo. Los principales riesgos y pasos manuales del release deben estar visibles, aunque algunos detalles todavía necesiten validación.

Las incertidumbres pendientes deben ser explícitas. Tenemos que distinguir lo que debe resolverse antes del release de lo que podemos aprender con seguridad durante la implementación, y acordar qué nos haría parar o cambiar de dirección. Eso es más útil que preguntar si todo el mundo se siente seguro.

Antes de dedicar más tiempo a un plan, quiero preguntar: *"¿Qué decisión cambiará el siguiente paso de planificación, y por qué planificar es la mejor forma de responder a esa pregunta?"*

Cuando podemos señalar un supuesto que saldría caro si fuera incorrecto y una forma útil de comprobarlo, merece la pena seguir planificando. Cuando la siguiente respuesta necesita código funcionando, toca construir la parte más pequeña que pueda darnos esa respuesta y llevar lo aprendido de vuelta al plan.